

LAB SESSION 10

Implementation of Depth-First Search (DFS) Algorithm

Objective

- To understand the concept of Depth-First Search (DFS).
- To implement DFS using recursion and stack.
- To traverse a graph depth-wise.
- To analyze the time and space complexity of DFS.

Introduction

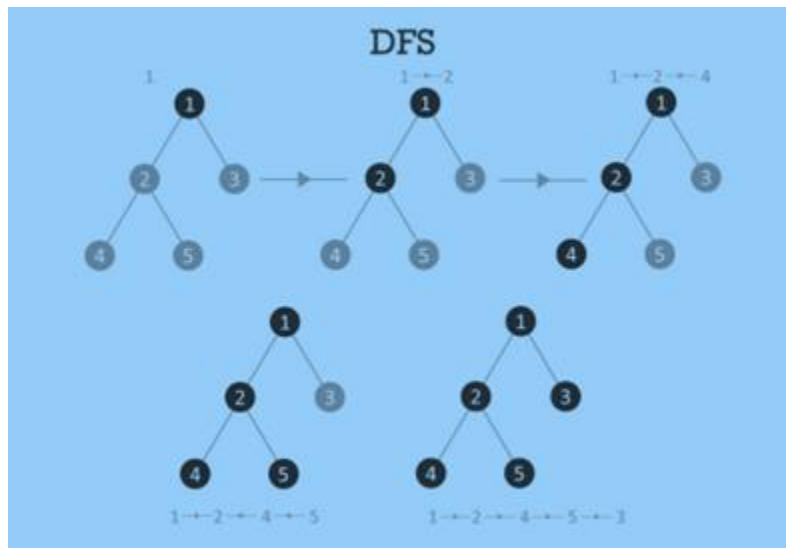
Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along one branch before backtracking.

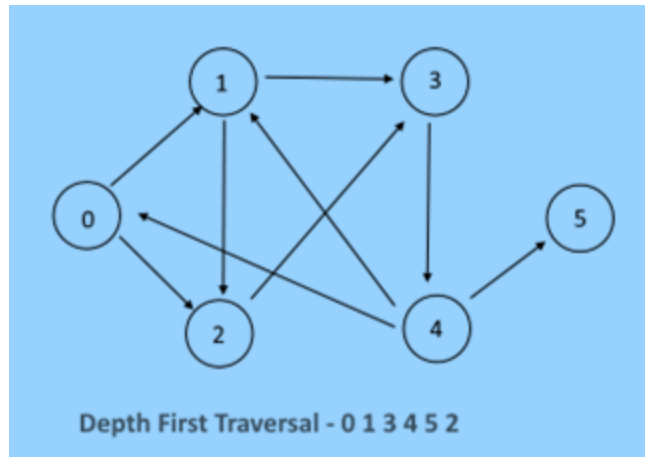
Unlike BFS (which works level-by-level), DFS goes deep first, then comes back.

DFS uses:

- Stack (LIFO – Last In First Out)
OR
- Recursion (implicit stack)

Graph Example





If we start DFS from node **0**, traversal order may be:

0 → 1 → 3 → 4 → 2 → 5

(DFS order may vary depending on adjacency list order.)

Algorithm (Recursive Method)

1. Start from source node.
2. Mark it as visited.
3. Print the node.
4. For each adjacent node:
 - If not visited → Recursively call DFS.

Pseudocode

```
DFS(G, node):
    mark node as visited
    print node

    for each neighbor of node:
        if neighbor not visited:
            DFS(G, neighbor)
```

C++ Program (Recursive DFS)

```
#include <iostream>
#include <vector>
using namespace std;

void DFS(vector<vector<int>>& graph, vector<bool>& visited, int node) {
    visited[node] = true;
    cout << node << " ";
```

```

        for(int neighbor : graph[node]) {
            if(!visited[neighbor]) {
                DFS(graph, visited, neighbor);
            }
        }
    }

int main() {
    vector<vector<int>> graph = {
        {1,2},
        {0,3,4},
        {0,5},
        {1},
        {1},
        {2}
    };

    vector<bool> visited(graph.size(), false);

    cout << "DFS Traversal: ";
    DFS(graph, visited, 0);

    return 0;
}

```

Sample Output

DFS Traversal: 0 1 3 4 2 5

(Note: Order may change depending on adjacency list.)

Complexity

Complexity Type	Value
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

Where:

- V = Number of vertices
- E = Number of edges

Applications of DFS

- Detecting cycles in graph
- Topological sorting
- Solving maze problems
- Path finding
- Finding connected components

Task 1:

Modify the DFS program to:

- Take graph input from the user.
- Allow user to enter starting vertex.
- Perform DFS traversal.

Task 2:

Modify DFS to:

- Count total number of connected components in a graph.
(Hint: Run DFS for every unvisited vertex.)